Module 2: Water Quality for bull trout reintroduction

2026-09-24

Introduction

Water is a valuable resource that is important for maintaining many aspects of a community's well being. "Water quality" captures many components of the chemistry, biology, and physics of fresh water. In this module, we will study data about two of these components, water temperature (T) and dissolved oxygen (DO).

Bull trout are a culturally important, but threatened species of fish native to the U.S. Northwest and the Canadian Rockies. They provide an example of how water quality affects community culture, because, to thrive, bull trout require water temperatures below 14 degrees C and dissolved oxygen levels above 6.2 mg/L. Such cold clean water conditions are growing scarcer due to climate warming and pollution.

It is also important to note that water temperature and levels of dissolved oxygen are related. As water temperature increases, dissolved oxygen levels decrease in water. This is concerning as climate warming has the power to increase water temperature, causing dissolved oxygen to dip to unhealthy levels for aquatic life.

In this module, we will use data about water quality trends to help K'avi Fish and Wildlife managers decide which of two streams would be best for a reintroduction of bull trout on tribal land. The tribe only has funding to bring the bull trout back to one stream. They have narrowed the choice of possible locations for the reintroduction to two streams, where temperature and D.O. have been monitored for about 15 years. Neither site is perfect, but they are both pretty good options. Your job is to analyze the temperature and D.O. data to help the tribe decide which is the best site.

The upstream source of Stream A is a groundwater spring. Water in this stream is very clean and temperatures are steady because groundwater temperatures tend to match average yearly temperature. The upstream source of Stream B is an alpine lake that is fed by a glacier. The glacial melt-water is very cold and clean, but the glacier is shrinking as climate warms, so the cold-water input to the lake is growing smaller over time.

Learning Objectives for Module 2:

1. Apply foundational exploration and plotting skills from Module 1 to a new environmental dataset, and learn why structured "tidy" data is easier to work with
2. Practice troubleshooting errors and and dealing with missing information in real-world measurements
3. Create "time-series" plots to track environmental changes over multiple years
4. Explore how water quality data can help in decision making to protect culturally important species

---

INSTRUCTOR'S VERSION: Learning goals and instructor resources

We will reach these goals in six steps

1. On day 1 of this module, we'll start by reviewing the usual steps for getting started: setting the working directory, bringing a dataset into RStudio, etc.

2. Next, we'll "wrangle" the dataset into a format that will make it easier to analyze in R. This includes organizing columns of data in a "tidy" way, merging two datasets, changing the names of variables, etc. Data wrangling is an important component of data science. It's a way of making sure the data you analyze is in a format that is reliable and complete. Students will use new commands, like `pivot_longer()` and `full_join()` to make the data "tidy", so it can be more easily reliable. There are many websites that can help you and your students learn these new tools and approaches. Here are a few to get you started: tidying data (https://cran.r-project.org/web/packages/tidyr/vignettes/tidy-data.html); pivoting data (https://tidyr.tidyverse.org/articles/pivot.html); joining data (https://r4ds.hadley.nz/joins.html).

3. NEW SKILL: students will learn how to handle common code bugs, such as NAs.

4. After step 2, we will have the data in tidy form, and we can use familiar code to learn the summary statistics: means, minima, maxima, etc. We'll end Day 1 of the module using these statistics to make a preliminary assessment of whether Stream A or Stream B would be a better choice for reintroducing bull trout to K'avi land.

5. We will spend Day 2 analyzing patterns in the temperature and DO data from the two streams in more detail. First, we'll use familiar code to make a scatter plot of temperature and DO. This will help us visualize all of the data from both streams, which might give us insights we didn't see from the summary statistics. We can use this scatterplot to learn which stream had more years in the "danger zone" for bull trout, where temperatures were above 14 degrees C or dissolved oxygen was below 6.2 mg/L.

6. Next, we'll learn how to adapt our plotting code to show trends in temperature and dissolved oxygen over the 15 years that the K'avi have been collecting data from these streams. These "time series graphs" tell us whether water quality in the two streams is getting better, getting worse, or staying the same. This shows the adaptability of ggplot, and the ability to add graphical features. Here's a link with more information (https://ggplot2.tidyverse.org/)

7. Finally, you can use all the analyses you have conducted (summary statistics, scatter plots, and time series plots) to make recommendations to the K'avi fisheries managers about which stream is a better bet for supporting healthy bull trout populations on tribal land. How does each analysis help you understand patterns in water quality? Which analysis was most useful?

8. From the scientific analysis perspective, this module highlights why it is useful to visualizing data in multiple ways. There is one question: "Which of two streams has water quality most appropriate to maintain a healthy bull trout population?" But we ask it three times, after seeing summary statistics, then a scatter plot, then time series plots. The summary statistics and the scatter plot show that Stream B generally has better water quality than Stream A, but the time-series graph shows that both temperature and dissolved oxygen are getting worse for bull trout. The question of which stream is a better choice (the one with better values or the one without a dangerous trend) is for the class to discuss.

DAY 1: Loading, merging, and tidying data

Lesson 2.1: Review loading and previewing data

Preliminary steps

Like we did in Module 1, before we being to analyze our data, we need to do a few steps first.

Clearing the Environment

Before we start to do anything in R, we should clear our environment. As a reminder, your environment is where R stores all the objects and data you were working with since the last time you worked in R. See the code chunk below on how to clear your environment.

```
rm(list = ls())
```

Setting the Working Directory

We will now set our working directory. As you may remember, setting our working directory tells R where to find the files we want to work with on our computer.

NOTE: if using an R project, can skip this step because this markdown file should be in the project with the data.

```
current_path = rstudioapi::getActiveDocumentContext()$path
setwd(dirname(current_path))
```

Load in packages

Next, we will load in packages, to get the extra tools we need for this module loaded in R. For this module, we will load in the tidyverse package.

```
# install package if required
if(!require("tidyverse")) install.packages("tidyverse")
```

```
## Loading required package: tidyverse
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✔ dplyr     1.2.1     ✔ readr     2.2.0
## ✔ forcats   1.0.1     ✔ stringr   1.6.0
## ✔ ggplot2   4.0.3     ✔ tibble    3.3.1
## ✔ lubridate 1.9.5     ✔ tidyr     1.3.2
## ✔ purrr     1.2.2
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts
to become errors
```

```
# load package
library(tidyverse)
```

Load in our data

Now it's time to load in our data! In this scenario, water temperature data and dissolved oxygen data were collected and stored in two separate files. Therefore, we will need to load in both files in order to use them.

As a note, the units for temperature are Celsius (C) and the units for dissolved oxygen are mg/L.

```
stream_temp <- read.csv("streams_temperature.csv", header = TRUE, sep =",")

stream_DO <- read.csv("streams_DissolvedOxygen.csv", header = TRUE, sep =",")
```

Viewing the data

Now that our data is loaded into R, we can explore the data using the view and head commands. Our goal is to make sure things loaded correctly and to get a first look to see if the data make sense.

```
#View(stream_temp)
```

```
head(stream_temp)
```

```
##   year StreamA StreamB
## 1 2007   13.20   10.20
## 2 2008   12.50      NA
## 3 2009   13.90      NA
## 4 2010   11.36   11.67
## 5 2011   12.97      NA
## 6 2012   13.10   11.10
```

```
#View(stream_DO)
```

```
head(stream_DO)
```

```
##   year StreamA StreamB
## 1 2007    7.18      NA
## 2 2008    7.27    8.07
## 3 2009    7.46    7.26
## 4 2010    7.00      NA
## 5 2011    6.53      NA
## 6 2012    7.50    9.50
```

Class Question:

- What do you notice is different between using the command "View" and the command "head"? Which one do you think is more useful?
- Recalling the previous module, what does the dim() function tell us about a data frame?

```
dim(stream_temp)
```

```
## [1] 16  3
```

```
dim(stream_DO)
```

```
## [1] 16  3
```

Class Questions:

- How many data entries are in this dataset for water temperature?
- How many data entries are in this dataset for dissolved oxygen?

Lesson 2.1 Recap:

You now reviewed ways to set up your R environment, read in data, and explore your data. These steps are all important for when we analyze our data later in this module.

- loading a package using `library()`
- loading a CSV data file
- previewing the data frame using `View()`, `head()`, and `dim()`

Lesson 2.2: Tidying and joining data

As of now, our data for water temperature for Stream A and B exists in a separate data frame from the dissolved oxygen concentrations for Stream A and B. For K'avi Fish and Wildlife managers to truly understand what's going on, they need to analyze water temperature and dissolved oxygen together. To do this, we need to change how our data is set up in each data frame and then merge these data frames together.

Manipulating data so that it is formatted in a way that we can easily and reliably analyze is called "data wrangling". In R, we like our data to be "tidy". This means we don't want duplicate columns for the same variable. For example, in the stream_temp dataset, we have two columns for temperature, because we have a column for the water temperature of Stream A and the water temperature of Stream B. In a "tidy" version of thus dataset, we want all the temperature values to be in the same column.

If you want to learn more about tidy data, here's a useful website (https://cran.r-project.org/web/packages/tidyr/vignettes/tidy-data.html).

Tidying data

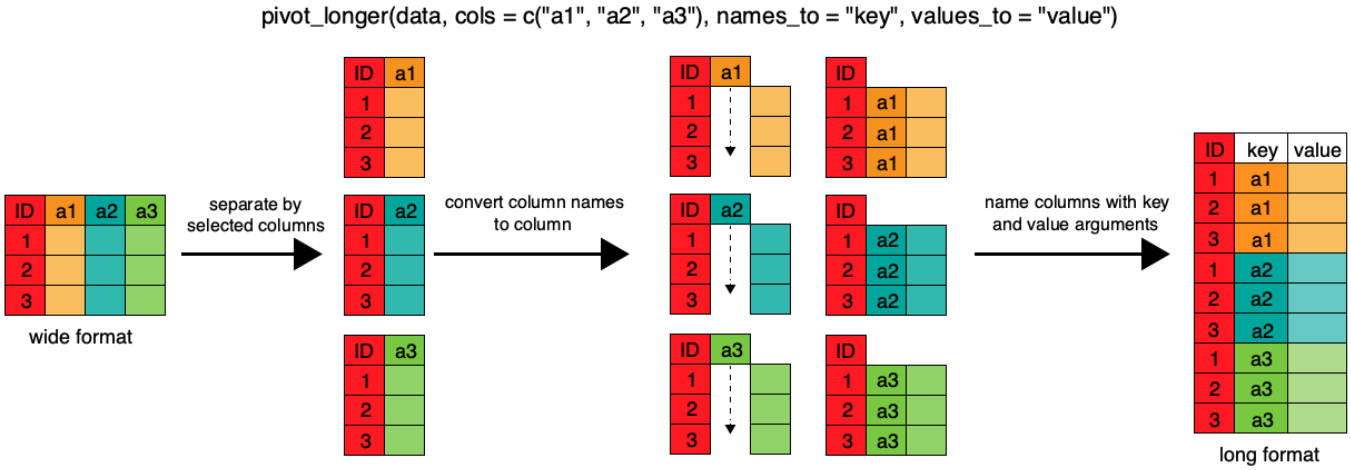
First, we will use the `pivot_longer()` command to reorganize the columns from the temperature dataset, so that there is one column for "site" and one column for "temperature_C" The goal is to make only one column for each variable.

Class Question

- Look back at the current "messy" version of stream_temp. Can you see that there is more than one column with the same variable?

The `pivot_longer()` command allows us to combine columns, which transforms our dataset into a form with more rows (longer), and fewer columns (less wide). Long form data makes it easier to summarize data and look at trends over time, which allows the tidyverse package to work better.

Here is a visual example:



In the code chunk below, we use `pivot_longer()` to transform the temperature data. The #comments describe what each line of code does. We rename our data frame to a new object with the suffix `_long` and then preview it with `head()`.

```
# The first line of code tells R that we want to re-define
# the streams_temp dataset with the changes we make below.
# This weird thing %>% is called the "pipe". It says that the code lines that come next,
# which are linked by a "+" at the end of each line,
# will all contribute to a transfomed version of stream_temp.

stream_temp_long <- stream_temp %>%
# We use the pivot_longer code to tell R which columns we want to change.
  pivot_longer(
    #cols = tells R which columns we want to modify
    cols = c(StreamA, StreamB),
    # names_to tells R that we want the names of the columns we specified
    # above to be entries in a new column we're creating called "site".
    names_to = "site",
    # Question: Based on what "names_to" means, what do you think the next line
    # of code values_to means?
    # (Hint - searching the internet for the answer is a great way to figure this out.)
    # Looking things up online is a huge part of coding!
    values_to = "temperature_C"
  )
```

To make sure this long command worked the way we intended, we should now view streams_temp:

```
print(stream_temp_long)
```

```
## # A tibble: 32 × 3
##     year site    temperature_C
##    <int> <chr>           <dbl>
##  1  2007 StreamA          13.2
##  2  2007 StreamB          10.2
##  3  2008 StreamA          12.5
##  4  2008 StreamB          NA
##  5  2009 StreamA          13.9
##  6  2009 StreamB          NA
##  7  2010 StreamA          11.4
##  8  2010 StreamB          11.7
##  9  2011 StreamA          13.0
## 10  2011 StreamB          NA
## # ℹ 22 more rows
```

Great! Our code worked. Now let's do the same for the dissolved oxygen data set.We want to change it in the same way we changed the temperature data set above.

💻 HOWEVER, this time some chunks of code have been left blank. See if you can fill them in based on what you saw in the example above!

```
stream_DO_long <- stream_DO %>%
# We use the pivot_longer code to tell R which columns we want to change.
  pivot_longer(
    # [FILL THIS IN]
    cols = c(StreamA, StreamB),
    # names_to tells R that we want the names of the columns we specified
    # above to be entries in a new column we're creating called "site".
    names_to = "site",
    values_to = "dissolved_oxygen"
  )
```

Again, we should check our work to make sure that our data looks the way we want it to. We want there to be three columns, one for the year the sample was collected, one for the site name, and one for the dissolved oxygen values (mg/L).

```
print(stream_DO_long)
```

```
## # A tibble: 32 × 3
##     year site    dissolved_oxygen
##    <int> <chr>              <dbl>
##  1  2007 StreamA             7.18
##  2  2007 StreamB            NA
##  3  2008 StreamA             7.27
##  4  2008 StreamB             8.07
##  5  2009 StreamA             7.46
##  6  2009 StreamB             7.26
##  7  2010 StreamA             7
##  8  2010 StreamB            NA
##  9  2011 StreamA             6.53
## 10  2011 StreamB            NA
## # ℹ 22 more rows
```

Good work! This data frame should have a single column of dissolved_oxygen and a single column of site to look the way we want it to.

Joining dataframes

So far, we have separate dataframes for temperature and DO. We want to combine them into one tidy dataframe, so we can analyze trends in temperature and dissolved oxygen at the same time for both streams.

To do this, we will use a command in tidyverse called `full_join()`, from the join commands. Full join means we want to combine two data frames. We want to match up the rows in the first data frame with the second data frame based on a matching key that the two have in common - in our case the year and site. See more on the join commands here (https://dplyr.tidyverse.org/reference/mutate-joins.html#inner-join).

The format for joining is: `full_join(dataframe1, dataframe2, by=c(columns they have in common))`

```
# Here, we are using full_join to put the streams_temp data frame next to the streams_DO data frame.
streams_joined <- full_join(stream_temp_long, stream_DO_long, by=c("year","site"))
```

Great! Now let's check to make sure that we successfully merged our two data frames.

```
print(streams_joined)
```

```
## # A tibble: 32 × 4
##     year site    temperature_C dissolved_oxygen
##    <int> <chr>           <dbl>            <dbl>
##  1  2007 StreamA          13.2             7.18
##  2  2007 StreamB          10.2            NA
##  3  2008 StreamA          12.5             7.27
##  4  2008 StreamB          NA               8.07
##  5  2009 StreamA          13.9             7.46
##  6  2009 StreamB          NA               7.26
##  7  2010 StreamA          11.4             7
##  8  2010 StreamB          11.7            NA
##  9  2011 StreamA          13.0             6.53
## 10  2011 StreamB          NA              NA
## # ℹ 22 more rows
```

Great job data wrangling! We now have a single tidy dataframe that we can use to analyze water quality and determine which stream is best for introducing bull trout to tribal land.

Now that we have a data frame in the format we want, it's good practice to save the dataframe so that you don't need to repeat the steps from above. We will do this using the write.csv() function, which takes two main inputs: the data frame we are saving, and the file name with location.

```
# save our tidy data frame for later
write.csv(streams_joined, "streams_data.csv", row.names = FALSE)
```

Lesson 2.2 Recap:

In this lesson we have learned the following:

- wrangling data into a tidy format, with one observation per row
- using `pivot_longer()` to reduce multiple columns into two columns with a key and a value. In our case, the stream ID was the key, and the observation was the temperature or DO value
- joining two data frames using `full_join()`, which combines dataframes using identifying columns that both data frames have in common - in our case, year and site

---

Lesson 2.3 Descriptive statistics and summarizing by group

Dealing with NAs

We know that bull trout prefer cold clean water. Temperatures above 14 degrees C and dissolved oxygen levels below 6.2 mg/L tend to be bad for bull trout, though these are only general approximations and the healthiness of stream water for bull trout depends on many factors.

We'll start our analysis of water quality by using the statistical summary commands that will be familiar from Module 1. First, we'll calculate the mean (or average) water temperature

```
# load in csv if needed
streams = read.csv("streams_data.csv")

mean(streams$temperature_C)
```

```
## [1] NA
```

Ah! Our code didn't work. R gave us an `<NA>` instead of an output. Not getting an expected output, or even error messages, are a normal part of programming. Although they might seem surprising or frustrating, you can think about them as a challenge for you to figure out.

Here, R is telling you that the dataset has missing data, which is indicated in R by the `<NA>` , which means "not available". As you may have already noticed when looking at the "streams" dataset, you'll see that there are, indeed, some NA values.

It is very common to have missing data in datasets. Sometimes the data contains NAs in them by the scientists that entered the data (as in our case), but R will also enter NA in a row if there was no data there to begin with (it was left blank when the data was collected).

The reason this causes R to throw an error message is because the command mean() assumes that there are actual numbers in each cell that R is computing the mean of. R is telling you that it doesn't know how to calculate the value of something called "NA".

In the next section, we'll go over some easy ways to fix this problem in our "streams" data set by telling R to ignore the NAs.

The command na.omit() deletes any row that has NAs in it. The code below doesn't change the streams dataset, it just prints streams without any rows that have NAs.

```
na.omit(streams)
```

```
##    year    site temperature_C dissolved_oxygen
## 1  2007 StreamA         13.20             7.18
## 3  2008 StreamA         12.50             7.27
## 5  2009 StreamA         13.90             7.46
## 7  2010 StreamA         11.36             7.00
## 9  2011 StreamA         12.97             6.53
## 11 2012 StreamA         13.10             7.50
## 12 2012 StreamB         11.10             9.50
## 13 2013 StreamA         14.47             6.16
## 14 2013 StreamB         11.42             8.78
## 15 2014 StreamA         13.37             7.05
## 16 2014 StreamB         11.37             8.35
## 17 2015 StreamA         12.48             7.34
## 18 2015 StreamB         12.18             8.44
## 19 2016 StreamA         12.63             6.15
## 20 2016 StreamB         12.03             8.23
## 21 2017 StreamA         13.16             6.89
## 22 2017 StreamB         11.96             8.28
## 23 2018 StreamA         12.94             6.03
## 24 2018 StreamB         12.56             7.83
## 25 2019 StreamA         14.10             7.26
## 26 2019 StreamB         13.00             7.01
## 27 2020 StreamA         12.07             7.53
## 28 2020 StreamB         12.87             7.30
## 29 2021 StreamA         13.43             7.02
## 30 2021 StreamB         13.25             7.42
## 31 2022 StreamA         13.28             6.32
## 32 2022 StreamB         13.78             6.63
```

Great! Now let's try to run our command to get the mean for temperature again. We don't want to remove these rows from the streams data permanently, so we'll nest the na.omit() command within the mean() command, allowing us to calculate the mean of all the temperatures for both streams,

```
mean(na.omit(streams$temperature_C))
```

```
## [1] 12.63276
```

Our code worked! Now you've learned how to handle NAs in a dataset in R. Let's carry on with our summary statistics.

 *Your turn!* How would you figure out what the mean amount of dissolved oxygen is in the water for both Stream A and Stream B?

```
#Write your code here!
```

Now lets use another familiar command to show the MEDIAN of temperature measured across all streams and years?

```
median(na.omit(streams$temperature_C))
```

```
## [1] 12.87
```

Class question:

What is the median amount of dissolved oxygen across all samples? 💻 Try and fill in the code chunk below:

```
#Write your code here!
```

Summarizing by group

We previously used `summary()` to get the descriptive statistics of a column or data frame, but this groups all the rows together. Since our goal is to compare the water quality of Stream A and Stream B, we want to get descriptive statistics for them separately. We can do this using `group_by()` and `summarize()`. We explain the code and the commands we will use in the code chunk below.

Read more about the summarize function from the dplyr package in R here. (https://dplyr.tidyverse.org/reference/summarise.html)

```
streams %>%
    # the column that includes our categorical group, site, is input to 'group_by()'
    group_by(site) %>%
    # we then use summarize() and input any function to apply to a column. In these exam
ples, all the same statistics from the 'summary()' function are applied to temperature_C
    summarize(min_temp = min(temperature_C, na.rm=T),
              first_q_temp = quantile(temperature_C, c(0.25), na.rm=T),
              median_temp = median(temperature_C, na.rm=T),
              mean_temp = mean(temperature_C, na.rm=T),
              third_q_temp = quantile(temperature_C, c(0.75), na.rm=T),
              max_temp = max(temperature_C, na.rm=T))
```

```
## # A tibble: 2 × 7
##   site    min_temp first_q_temp median_temp mean_temp third_q_temp max_temp
##   <chr>      <dbl>        <dbl>       <dbl>     <dbl>        <dbl>    <dbl>
## 1 StreamA     11.4         12.6        13.1      13.1         13.4     14.5
## 2 StreamB     10.2         11.4        12.0      12.1         12.9     13.8
```

💻 Your turn! You can add columns to show descriptive statistics of dissolved oxygen, or redo the above code with `dissolved_oxygen` instead of `temperature_C` inside the descriptive statistic functions.

```
# get descriptive statistics of dissolved oxygen for both sites
```

Preliminary analysis of water quality data from Stream A and Stream B

On Day 2 of this module, we will explore the water quality data in detail, and use this exploration of data to help inform the best decision about which stream is most promising for bull trout reintroduction. As a reminder, temperatures above 14 degrees C and dissolved oxygen levels below 6.2 mg/L tend to be bad for bull trout.

For now, let's use the information in the summary tables above to get a preliminary sense of which stream might have better water quality for bull trout.

Class Questions

- From what we've learned about the streams so far, which one do you think would be better to re-introduce bull trout into?
- Which of the summary statistics do you think is most useful?
- Is there any other information you'd like to know in order to make the best recommendation to the tribe?

Lesson 2.3 Recap:

In this section, we reviewed - how to remove NAs - how to run summary statistics by group, using `group_by()` and `summarize()`

DAY 2: Detailed exploration of the water quality data with graphs.

The descriptive statistics we generated using the summary() command gave us an overview of how water temperature and dissolved oxygen differ between the two sites. Summary statistics are useful, and tribes are often asked to generate such statistics for reports (for instance the water quality reports tribes for the U.S. Environmental Protection Agency). However, it is always a good idea to look at your data in many ways, including charts and graphs. Today, we will look at two different graphs of the water quality data, then revisit the question of which stream would be best for the bull trout reintroduction.

Lesson 2.4: Exploring water quality data with graphs

Scatter plots

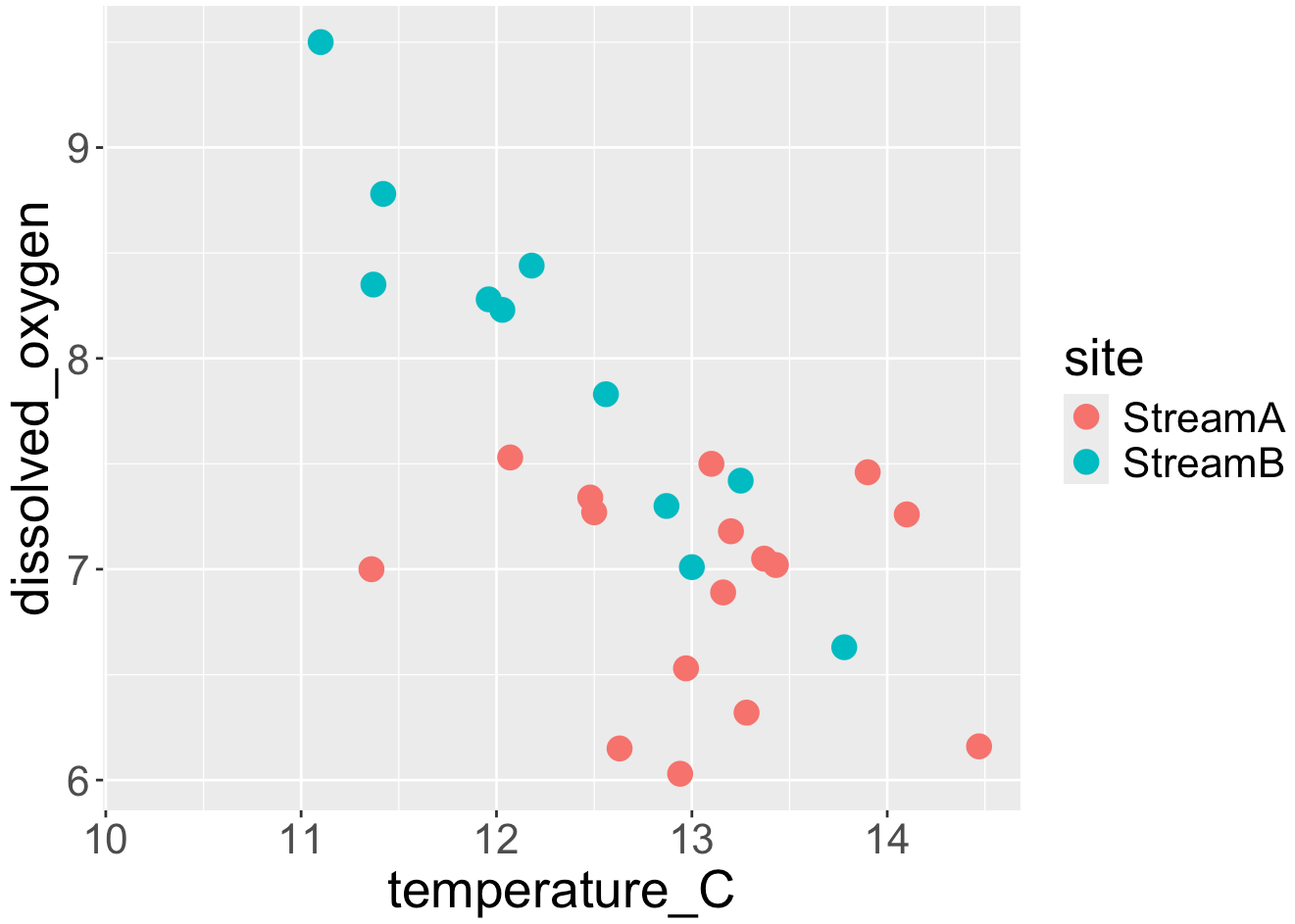
Bull trout prefer water below 14 deg C and above 6.2 mg/L dissolved oxygen, but these thresholds are only part of the story. We can use a scatter plot to look at all the data for both streams to see patterns in the overall temperature and DO values are.

We'll use code that is similar to the scatter plot code in Module 1.

```
# The first line says we're making a plot using the streams dataframe.
# We're going to plot temperature_C against dissolved_oxygen.
# And we're going to color the symbols by site, distinguishing StreamA and StreamB.
# 'size' adjusts the size of the points to be larger
# we are saving it as an object called 'scatter_plot'
scatter_plot <- ggplot(streams, aes(temperature_C, dissolved_oxygen, color = site)) +
  # Then we specify a scatter plot (points) and increase size of points
  geom_point(size=4) +
  # increase size of text
  theme(text=element_text(size=20))

# see output
scatter_plot
```

```
## Warning: Removed 5 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

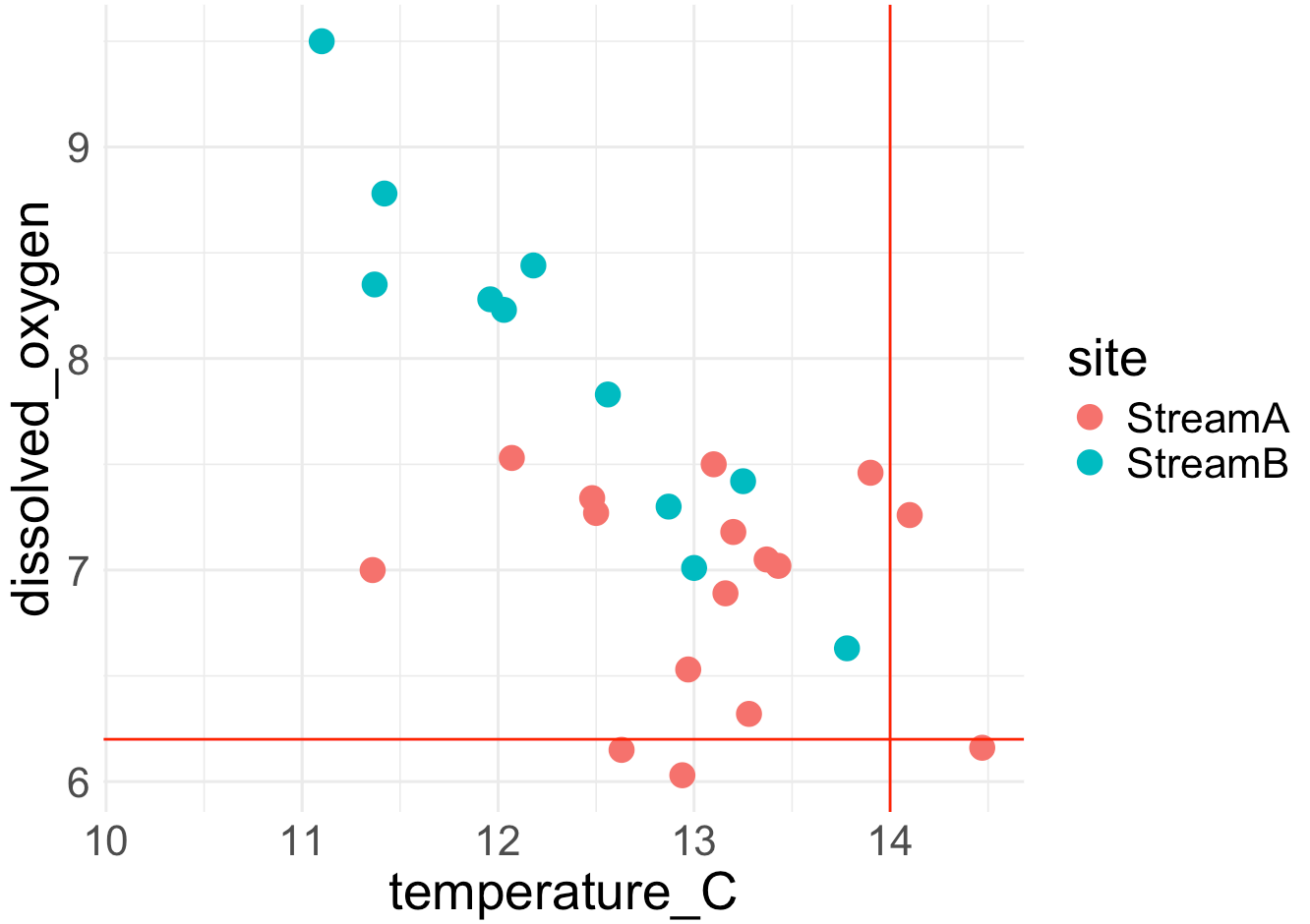


We can add markers in our plot to show how many points fall outside our water quality thresholds for dissolved oxygen and temperature. We do this using the ggplot geoms:

- `geom_hline(yintercept)`
- `geom_vline(xintercept)`
- like any other geoms, we can also change the setting of color, in this case we change it to red for both lines

```
# for reference we can add red lines depicting the levels of temperature and DO that are dangerous for bull trout.
# because we saved it as an object, we can add to the object directly
scatter_plot +
  geom_hline(yintercept = 6.2, col = "red") +
  geom_vline(xintercept = 14, col = "red") +
  # We'll use the same simple "theme" we used in module 1.
  theme_minimal() +
  # We will also make the text larger so it's easier to read.
  theme(text=element_text(size=20))
```

```
## Warning: Removed 5 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



Class Questions:

- Where are the best values of water quality for bull trout? Where are the worst values?
- When you ran the code chunk, you might have gotten a warning message that says, "Warning: Removed 5 rows containing missing values (`geom_point()`)." What does that mean? How does this affect your interpretation of the graph?
- Does looking at this graph change your opinion about whether Stream A or Stream B will be healthier for bull trout? What other information would help the tribe make this decision?

Time series plots

The scatter plot shows more detail than the summary statistics, but there is still more useful information in our data set. K'avi Fish and Wildlife managers have measured temperature and DO for 15 years, which is a long time. Since we know that temperatures are rising, it would be useful to see if these values change over time (we're looking for a trend).

To recap, Stream A is fed by a groundwater spring, and Stream B is fed by an alpine lake that is fed by a glacier. Because there sources of water are different, we can hypothesize that these streams may react differently to climate change.

Class Question:

- Based on what we know about Stream A and Stream B, what would your hypothesis be on how the water quality for each stream may change as global temperatures increase with climate change?

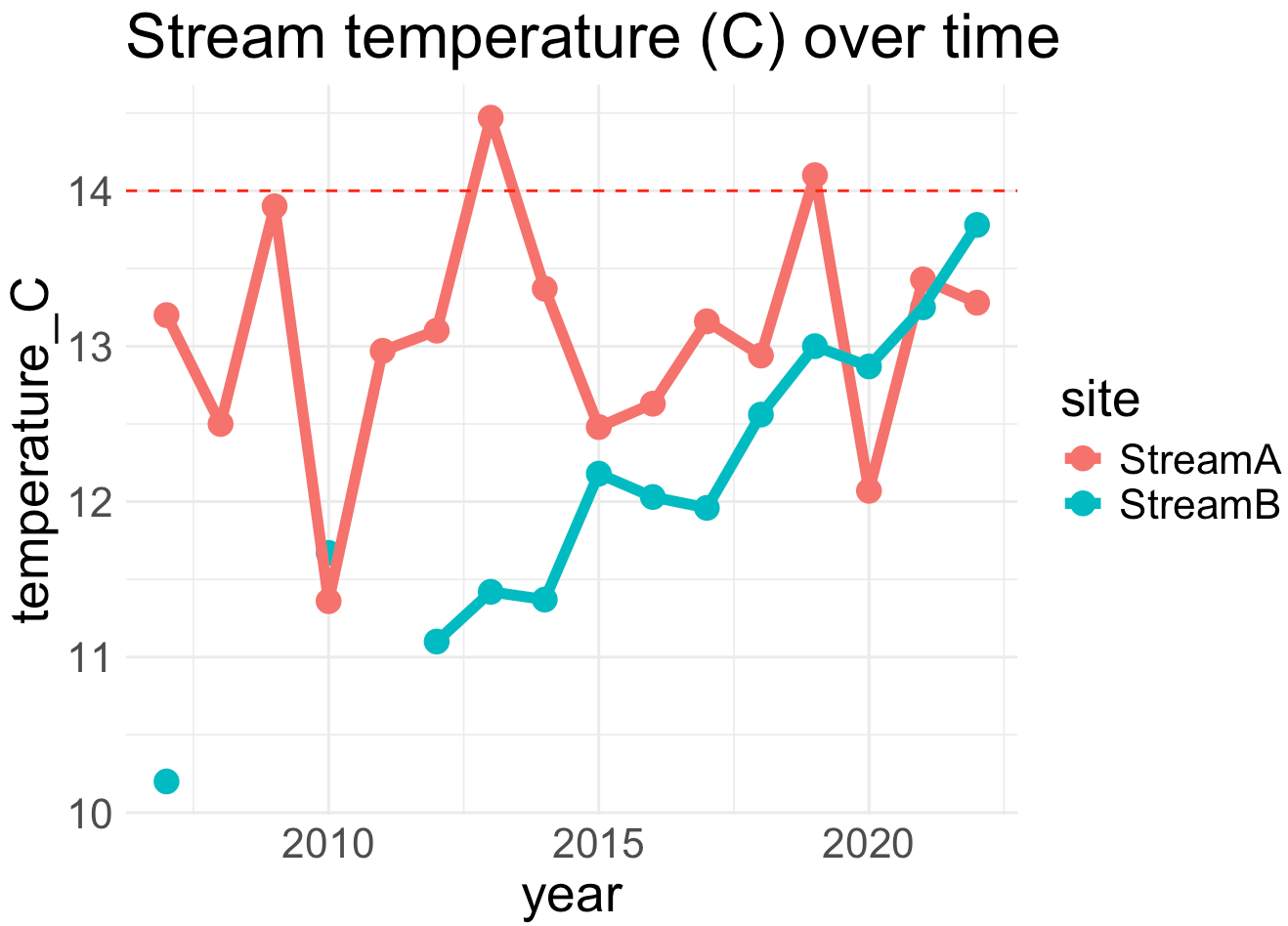
We can assess our hypotheses by plotting water temperature and dissolved oxygen over time. We'll make a small change to our typical ggplot() code to make a "time series" plot, which shows trends across time.

Here we will use `geom_line()` to plot temperatures from both sites across the 15 years that the tribe has been monitoring water temperatures. Similar to the scatter plot, we can assign the site column to color.

```
# The first line says we're making a plot using the streams dataframe.
# We're going to plot temperature_C over time (year).
# And we're going to color the symbols by site, distinguishing StreamA and StreamB.
ggplot(streams, aes(x=year, y=temperature_C, color = site)) +
  # We start by plotting points (as we did in the scatter plot code)
  geom_point(size=4) +
  # Next, we add lines connecting adjacent points in time using geom_line()
  # Note how ggplot() allows you to add features like lines in a step by step way.
  geom_line(size=2) +
  # We'll add a line indicating the "danger zone" of temperatures above 14 C.
  geom_hline(yintercept = 14, col = "red", linetype="dashed") +
  # and we'll use the same theme as always.
  theme_minimal() +
  # make text larger
  theme(text=element_text(size=20)) +
  # set a title
  ggtitle("Stream temperature (C) over time")
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## ℹ Please use `linewidth` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



Class Questions:

- What do the two colored lines represent?
- Why are there breaks in the blue line between 2008-2011?
- Do the trends over time support your hypothesis about the stability of the temperature of the streams? Why or why not?

💻 Your turn! Using the code above as an example, see if you can make a similar time-series plot showing changes in dissolved oxygen for both streams over time:

```
# Write your code here:
# hint: you'll need to change two variables – one for the line and points, and one for t
he horizontal line representing the threshold
```

Class Questions:

- Why do you think the water quality in Stream B is getting worse?
- Which stream do you think will be healthier for bull trout in the future?
- If you were using this analysis to make a recommendation to K'avi Fish and Wildlife managers, what would you suggest they consider as they decide which stream should be the site for bull trout reintroductions?
- Beyond the case of bull trout reintroductions, what other water quality issues are important to tribes?
- How can coding in R help inform tribal decisions about water quality?

Lesson 2.4 Recap:

In this lesson, we plotted our data using ggplot():

- a scatter plot of temperature and dissolved oxygen
- time series of both variables on separate plots
- for both plots, we grouped by stream using color